

SAPIENZA
UNIVERSITÀ DI ROMA

Sviluppo di un Framework di Machine Learning per l'Analisi e la Classificazione di Segnali da Sensori di Gas

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Corso di Laurea in Ingegneria Informatica e Automatica

Matteo Tesei
Matricola 2121619

Relatore
Prof. Giampaolo Liuzzi

Anno Accademico 2025/2026

Tesi non ancora discussa

Sviluppo di un Framework di Machine Learning per l'Analisi e la Classificazione di Segnali da Sensori di Gas

Relazione Tecnica Finale. Sapienza Università di Roma

© 2026 Matteo Tesei. Tutti i diritti riservati

Questa tesi è stata composta con LaTeX e la classe Sapthesis.

Email dell'autore: tesei.2121619@studenti.uniroma1.it

Sommario

Il rilevamento e la classificazione accurata di sostanze chimiche aerodisperse rappresentano una sfida critica in ambito industriale e della sicurezza, resa complessa dalla cross-sensitività e dalla forte non-linearità dei sensori gas commerciali. Il presente lavoro di tesi illustra la progettazione, l'ingegnerizzazione e la validazione di un *framework* software finalizzato all'elaborazione dei segnali provenienti da una matrice di sensori (MOX e capacitivi) fornita da Sensichips [1], con l'obiettivo di identificare 25 differenti classi chimiche e condizioni ambientali.

Per superare i limiti predittivi intrinseci riscontrati sui segnali sensoriali grezzi, è stata sviluppata un'articolata *pipeline* di *Data Engineering*. L'applicazione congiunta della sottrazione della *baseline* ambientale, di un algoritmo *custom* di bilanciamento ibrido delle classi (*Hybrid Resampling*) e di una logica di aggregazione per esperimento (*Majority Voting*), validata tramite *Group K-Fold*, ha permesso di stabilizzare la convergenza dei modelli. In questa configurazione *flat*, un Multi-Layer Perceptron (MLP) ad architettura profonda ha raggiunto un'accuratezza globale del 94.6% ($\pm$ 4.9%) sulla discriminazione dell'intero spettro chimico considerato.

Successivamente, al fine di rispondere a stringenti requisiti di sicurezza industriale e minimizzare i falsi negativi sulle sostanze infiammabili, il sistema è stato riprogettato secondo un'architettura gerarchica a due stadi. Un classificatore binario ad alto sbilanciamento (*LightGBM*) opera come filtro primario per isolare la macro-categoria degli idrocarburi, i quali vengono poi elaborati da un secondo modello specializzato. L'introduzione di una soglia di confidenza al 70% in questo secondo stadio ha fornito al sistema un robusto meccanismo di rigetto, capace di identificare con precisione i gas critici e di sopprimere efficacemente i falsi allarmi generati da interferenze ambientali.

L'intero sistema è stato implementato in linguaggio Python, seguendo i moderni standard di Ingegneria del Software (modularità, tipizzazione statica e configurazione centralizzata), configurandosi come uno strumento di laboratorio scalabile e pronto per l'integrazione su dispositivi *edge*.

Parole chiave: Machine Learning, Sensori di gas, Classificazione gerarchica, Sistemi safety-critical, Data Engineering.

Disponibilità del codice: Il codice sorgente del *framework*, comprensivo dei moduli di ETL, orchestrazione e reportistica, è liberamente consultabile alla seguente repository GitHub:
https://github.com/M4tt3o-T/Tesi

Indice

1 Introduzione **1**
1.1 Il problema della classificazione dei gas 1
1.2 Obiettivi della tesi 2
1.3 Struttura del documento 3

2 Definizione dei Requisiti **5**
2.1 Requisiti Funzionali 5
2.1.1 Acquisizione e Preprocessing dei Dati 5
2.1.2 Moduli di Classificazione (Flat e Gerarchici) 5
2.1.3 Logica di Valutazione per Esperimento (Majority Voting) . . 6
2.1.4 Reportistica e Logging 6
2.2 Requisiti Non Funzionali 6
2.2.1 Target di Esecuzione e Prestazioni 6
2.2.2 Portabilità e Indipendenza dalla Piattaforma 7
2.2.3 Usabilità e Configurabilità Centralizzata 7
2.2.4 Qualità del Codice e Manutenibilità 7

3 Design e Architettura del Sistema **9**
3.1 Architettura modulare della pipeline di Machine Learning 9
3.2 Il gestore dei dati: `DataManager` 10
3.3 Il motore di esecuzione: `ExperimentRunner` 11
3.3.1 L'aggregazione delle predizioni: Majority Voting 11
3.3.2 Orchestrazione della Pipeline Gerarchica 11

4 Dettagli Tecnologici e Implementazione **13**
4.1 Tecnologie utilizzate: Python e l'ecosistema Scikit-learn 13
4.1.1 Il ruolo di Scikit-learn 13
4.1.2 L'ecosistema di supporto: NumPy e Pandas 14
4.2 Algoritmi di classificazione: dai modelli classici a LightGBM e MLP 14
4.2.1 I limiti dei modelli classici e delle SVM 14
4.2.2 L'efficienza del Gradient Boosting e degli alberi decisionali . . 15
4.2.3 La predominanza del Multi-Layer Perceptron (MLP) 15
4.3 Implementazione del preprocessing e normalizzazione dei segnali . . 15
4.3.1 Sottrazione della Baseline Ambientale 16
4.3.2 Estrazione delle Feature 16
4.3.3 Scaling e Normalizzazione (StandardScaler) 16

4.4 Gestione della configurazione centralizzata: `config.py` 16
4.4.1 Definizione dell'Ambiente e dei Target Chimici 17
4.4.2 Iniezione delle Dipendenze (Dependency Injection) e Duck Typing 17
4.4.3 Progettazione Topologica per Reti Neurali 17

5 Sperimentazione e Validazione **19**
5.1 Protocollo sperimentale e K-Fold Cross Validation 19
5.1.1 La validazione incrociata e l'isolamento degli esperimenti . . 19
5.1.2 Strategia di Bilanciamento (Hybrid Resampling) 19
5.1.3 Metriche di Valutazione 20
5.2 Analisi dei risultati per modelli standard (Fase 1) 20
5.2.1 L'impatto della Pipeline di Preprocessing e del Majority Voting 21
5.2.2 La topologia ottimale: MLP (512, 256) 21
5.3 Valutazione delle prestazioni gerarchiche (Fase 2) 21
5.3.1 Stage 1: Il Filtro Binario (LightGBM) 22
5.3.2 Stage 2: Lo Specialista (MLP 64-256-64) 22
5.3.3 Meccanismo di Rigetto e Ottimizzazione della Soglia 22
5.4 Analisi visiva tramite Matrici di Confusione 22
5.4.1 Matrice di Confusione - Fase 1 (Classificazione Flat) 23
5.4.2 Matrice di Confusione - Fase 2 (Stage 2 Gerarchico) 24

6 Conclusioni e Sviluppi Futuri **25**
6.1 Sintesi del lavoro e raggiungimento degli obiettivi 25
6.2 Sviluppi Futuri 26

Bibliografia **27**

Capitolo 1

Introduzione

1.1 Il problema della classificazione dei gas

Negli ultimi anni, lo sviluppo di reti di sensori chimici miniaturizzati, spesso definiti come *Electronic Noses* (Nasi Elettronici [2]), ha aperto nuove frontiere nel campo del monitoraggio ambientale e della sicurezza industriale. L'obiettivo di questi dispositivi è rilevare e identificare composti organici volatili (VOC) e miscele di gas presenti nell'aria in diverse concentrazioni.

A differenza della strumentazione analitica tradizionale da laboratorio, come la gascromatografia o la spettrometria di massa (estremamente precise ma costose, ingombranti e lente), i sensori a stato solido offrono rilevazioni in tempo reale, bassi consumi energetici e dimensioni ridotte. Nel contesto di questo progetto, i dati analizzati provengono da un avanzato sensore per il monitoraggio dell'aria (SCA Air Sensor prodotto da Sensichips [1]) che integra due diverse tecnologie: un sensore a ossido di metallo (MOX) basato su un film di (SnO_2) e oro, e un sensore capacitivo basato su alluminio ossidato (Al_2O_3).

Per estrarre informazioni sufficienti a distinguere le molecole, il dispositivo non si limita a una lettura statica, ma adotta tecniche di modulazione dinamica, ampiamente documentate in letteratura [3], come la *Temperature Cycled Operation* (TCO) e la *Voltage Cycled Operation* (VCO). Il risultato di ogni campionamento è un array multidimensionale composto da 16 feature, derivate da misurazioni di impedenza in corrente alternata (componenti in fase e in quadratura) registrate a diverse frequenze (78 kHz e 200 Hz) e a diversi step di temperatura (da 150°C a 400°C).

Tuttavia, l'utilizzo di queste tecnologie introduce sfide algoritmiche notevoli. Il problema fondamentale nella classificazione dei gas tramite array di sensori è la cosiddetta *cross-sensitività*: i sensori chimici non sono perfettamente selettivi, ma tendono a reagire in modo sovrapposto a classi diverse di sostanze [4]. Inoltre, il segnale grezzo è fortemente influenzato da disturbi ambientali, quali variazioni di temperatura, livelli di umidità, turbolenze dell'aria e fluttuazioni della *baseline* del sensore nel tempo. A causa di questa spiccata non-linearità, i classici approcci basati su semplici soglie fisse risultano inefficaci, rendendo indispensabile l'impiego di tecniche moderne di Machine Learning e Deep Learning [5]. L'applicazione di questi paradigmi — le cui ramificazioni teoriche si estendono oggi fino all'apprendimento

per rinforzo e al controllo ottimo [6] — è infatti l'unica in grado di mappare i pattern complessi (le "impronte digitali" dei gas) estratti dalle matrici di sensori.

In questo scenario, emerge una criticità specifica legata al dominio applicativo della sicurezza: non tutte le sostanze rilevate presentano lo stesso livello di rischio. Distinguere l'evaporazione di aceto di mele o di un profumo (sostanze innocue) da una perdita di idrocarburi come metano, butano, benzina o cherosene rappresenta un requisito fondamentale. Gli idrocarburi sono composti altamente infiammabili ed esplosivi, e un falso negativo (la mancata rilevazione di un idrocarburo) o un'errata classificazione possono avere gravi ripercussioni in termini di sicurezza.

I classificatori standard "piatti" (*flat classifiers*) trattano tutte le classi allo stesso modo, massimizzando l'accuratezza globale ma rischiando di confondere una sostanza critica con una innocua in situazioni di incertezza o in presenza di rumore. Emerge pertanto la necessità di affrontare il problema della classificazione dei gas non come un unico task monolitico, ma attraverso un'architettura decisionale gerarchica, capace di isolare prioritariamente le sostanze ad alto rischio per poi demandarne l'identificazione specifica a modelli altamente specializzati e dotati di meccanismi di rigetto (soglie di confidenza).

1.2 Obiettivi della tesi

L'obiettivo principale di questo lavoro è la progettazione e lo sviluppo di una *pipeline* di Machine Learning completa, capace di elaborare i segnali provenienti dalla matrice di sensori e di classificarne correttamente la natura chimica. Affinché il sistema risulti robusto e applicabile in scenari reali, il progetto è stato strutturato in due macro-fasi consequenziali.

Nello specifico, gli obiettivi fondamentali si articolano come segue:

- **Fase 1 - Classificazione Multi-classe (Flat Classification):** il primo obiettivo consiste nello sviluppare algoritmi in grado di discriminare con elevata accuratezza tutte le 25 sostanze (composti chimici e condizioni ambientali) presenti nel dataset. Lo scopo di questa fase è dimostrare l'effettiva capacità dei sensori e delle tecniche di *preprocessing* di estrarre pattern distintivi per ogni singolo gas, stabilendo una solida *baseline* di performance su un problema a elevata cardinalità.

- **Fase 2 - Architettura Gerarchica per la Sicurezza:** a partire dai limiti intrinseci della classificazione standard (dove l'errore tra due gas innocui ha lo stesso peso matematico di un errore tra un gas innocuo e uno pericoloso), l'analisi si estende per affrontare le criticità individuate. Per rispondere a rigidi requisiti di sicurezza, è stato progettato un modello a due stadi finalizzato a massimizzare l'accuratezza nel riconoscimento degli idrocarburi. In questa fase, il sistema accetta una classificazione meno granulare per le sostanze a basso rischio, minimizzando rigorosamente i falsi negativi sulle molecole infiammabili tramite l'introduzione di soglie di confidenza (meccanismo di rigetto).

- **Ingegnerizzazione del Software:** oltre all'aspetto modellistico, un obiettivo trasversale è la realizzazione di un *framework* software aderente ai moderni

standard di sviluppo in Python. Il codice è stato strutturato secondo i principi di modularità e singola responsabilità, implementando un motore di validazione automatizzato e un modulo di configurazione centralizzato, così da fornire uno strumento manutenibile e scalabile.

1.3 Struttura del documento

Il presente lavoro è organizzato in sei capitoli che illustrano il percorso metodologico seguito, a partire dai presupposti tecnologici fino alla validazione sperimentale dell'architettura proposta:

- **Capitolo 1 - Introduzione:** descrive il contesto tecnologico dei sensori di gas, le sfide legate alla classificazione dei segnali chimici e definisce gli obiettivi della ricerca, con particolare enfasi sulla sicurezza industriale.
- **Capitolo 2 - Definizione dei Requisiti:** analizza le necessità funzionali e non funzionali del sistema, definendo i vincoli tecnici necessari per l'elaborazione dei dati e la robustezza dell'infrastruttura software.
- **Capitolo 3 - Design e Architettura del Sistema:** illustra la progettazione modulare del *framework*, descrivendo il ruolo dei principali componenti software e la logica decisionale alla base dell'architettura gerarchica a due stadi.
- **Capitolo 4 - Dettagli Tecnologici e Implementazione:** approfondisce le tecnologie utilizzate, le procedure di *preprocessing* e normalizzazione dei segnali e la configurazione degli algoritmi di Machine Learning impiegati.
- **Capitolo 5 - Sperimentazione e Validazione:** presenta i risultati dei test condotti, confrontando le prestazioni della classificazione *flat* con l'approccio gerarchico e analizzando l'efficacia delle soglie di confidenza tramite matrici di confusione.
- **Capitolo 6 - Conclusioni:** riassume i risultati ottenuti e discute le possibili evoluzioni del progetto e i futuri ambiti di ricerca.

Capitolo 2

Definizione dei Requisiti

Il presente capitolo definisce e formalizza le specifiche tecniche del sistema software sviluppato. La fase di ingegneria dei requisiti permette di mappare la complessità dei dati fisici e di garantire l'affidabilità del sistema negli scenari d'uso previsti, isolando le responsabilità logiche dai dettagli implementativi.

2.1 Requisiti Funzionali

I requisiti funzionali definiscono le operazioni e il comportamento atteso della *pipeline* di Machine Learning, coprendo l'intero flusso di lavoro, dall'acquisizione dei dati alla produzione dei risultati finali.

2.1.1 Acquisizione e Preprocessing dei Dati

Il sistema deve interfacciarsi con set di dati grezzi in formato CSV, derivanti dalle misurazioni dell'array di sensori (MOX e capacitivi). I requisiti specifici per questo modulo includono:

- **Lettura e Cache Dinamica:** capacità di scansionare automaticamente le directory degli esperimenti e ottimizzare le operazioni di I/O tramite la generazione di una cache binaria (formato `.npy`) dei tensori estratti.
- **Filtraggio e Normalizzazione:** il sistema deve implementare la sottrazione della *baseline* ambientale e l'estrazione di *feature* statistiche (media e varianza) per ogni esperimento, riducendo l'impatto del rumore di fondo.
- **Generazione Dataset:** partizionamento automatico dei dati in set di addestramento e test mediante *Group K-Fold*, garantendo che le istanze appartenenti a un singolo esperimento fisico non siano divise tra i set per prevenire fenomeni di *data leakage* [7]. Questa logica è fondamentale per evitare una sovrastima ottimistica delle reali capacità diagnostiche dei classificatori.

2.1.2 Moduli di Classificazione (Flat e Gerarchici)

Il nucleo algoritmico deve supportare l'addestramento e l'inferenza secondo due paradigmi distinti:

- **Classificazione Flat:** il software deve istanziare e addestrare classificatori multi-classe standard (es. Multi-Layer Perceptron, LightGBM, Random Forest) per la discriminazione simultanea delle 25 classi target.
- **Architettura Gerarchica a Due Stadi:** implementazione di una logica in cui un classificatore binario (Stage 1) isola la macro-categoria degli idrocarburi, delegando l'identificazione specifica a un modello specializzato (Stage 2).
- **Meccanismo di Rigetto:** lo Stage 2 deve applicare una soglia di confidenza dinamica. Se la probabilità predetta risulta inferiore a tale limite, il sistema deve retrocedere il campione, ri-etichettandolo come "Non-Idrocarburo" per ridurre i falsi positivi.

2.1.3 Logica di Valutazione per Esperimento (Majority Voting)

I dati sensoriali sono trattati come insiemi non ordinati di campioni indipendenti (*bag-of-frames*). Per consolidare le rilevazioni di un singolo esperimento, il sistema deve implementare una logica di *Majority Voting*: le predizioni sui singoli *frame* vengono aggregate per determinare la classe prevalente, restituendo un'unica etichetta globale per l'intera sessione di misura.

2.1.4 Reportistica e Logging

Al termine di ogni ciclo di test, il sistema deve produrre una documentazione dettagliata delle prestazioni. È richiesto il salvataggio di file di log con le metriche di *accuracy* e deviazione standard, l'individuazione delle migliori configurazioni iperparametriche e la generazione di grafici e matrici di confusione ad alta risoluzione per l'analisi diagnostica.

2.2 Requisiti Non Funzionali

I requisiti non funzionali non descrivono specifiche operazioni di calcolo, ma definiscono gli standard di qualità, robustezza e fruibilità del software. Essendo questo progetto concepito all'interno di un contesto accademico e di ricerca, l'ingegnerizzazione del codice ha richiesto un'attenzione pari a quella dedicata allo sviluppo algoritmico.

2.2.1 Target di Esecuzione e Prestazioni

Data l'elevata complessità computazionale richiesta dall'addestramento e dalla validazione incrociata (K-Fold) di modelli avanzati come Reti Neurali (MLP) o algoritmi di *Gradient Boosting* (LightGBM), il *framework* è stato progettato come strumento per la ricerca *offline* (elaborazione *batch*). L'esecuzione è pensata per essere ospitata su *workstation* standard. Non essendo previsti, in questa fase, vincoli applicativi di elaborazione *real-time* su dispositivi *edge* o a basso consumo (es. microcontrollori integrati nel sensore), il sistema privilegia la profondità dell'analisi e l'accuratezza diagnostica rispetto alla latenza di inferenza.

2.2.2 Portabilità e Indipendenza dalla Piattaforma

Il software deve garantire una totale portabilità tra diversi ambienti operativi. Sviluppato interamente in Python, il codice sorgente è agnostico rispetto al sistema operativo host. Il funzionamento e la corretta risoluzione dei percorsi di I/O sono stati esplicitamente validati sia in ambiente Windows (Windows 11) sia in ambiente Linux (Ubuntu 24.04). La replicabilità dell'ambiente di calcolo è inoltre garantita dall'isolamento delle dipendenze tramite un apposito file `requirements.txt`.

2.2.3 Usabilità e Configurabilità Centralizzata

Per consentire a ricercatori e operatori di laboratorio di condurre nuovi esperimenti senza dover intervenire sulla logica algoritmica, l'intera configurazione del sistema è stata scorporata e centralizzata nel modulo `config.py`. Questo garantisce un'usabilità elevata: l'inserimento di nuovi gas target, la modifica delle soglie di confidenza per lo Stage 2 (meccanismo di rigetto) o l'aggiornamento degli iperparametri delle reti neurali richiedono unicamente la modifica di variabili dichiarative, prevenendo il rischio di alterazioni accidentali ai moduli esecutivi.

2.2.4 Qualità del Codice e Manutenibilità

Il codice è stato sviluppato in rigorosa conformità agli standard industriali di Ingegneria del Software. Per massimizzare la leggibilità e la manutenibilità futura, sono state adottate le seguenti metodologie:

- **Type Hinting:** tipizzazione statica esplicita di tutte le funzioni, classi e firme dei metodi (PEP 484), al fine di facilitare l'analisi statica e ridurre gli errori a *runtime*.
- **Linting e Formattazione Automatica:** validazione continua tramite analizzatori statici (Flake8, Pylance) per l'individuazione preventiva di potenziali anomalie logiche e formattazione del codice sorgente tramite lo standard *Black*, garantendo piena aderenza alle linee guida PEP 8.
- **Design Pattern:** applicazione del principio di singola responsabilità (SRP), strutturando l'ETL (Estrazione, Trasformazione e Caricamento), l'orchestrazione degli esperimenti e il livello di presentazione (reportistica) in moduli rigidamente disaccoppiati.

Capitolo 3

Design e Architettura del Sistema

Questo capitolo illustra le scelte architetturali e i *design pattern* adottati per la costruzione del *framework*. La complessità intrinseca nell'elaborazione di serie storiche e nella gestione di modelli gerarchici ha reso necessaria la progettazione di un'architettura software altamente disaccoppiata, in cui il flusso dei dati è rigidamente separato dalle logiche di addestramento e di reportistica.

3.1 Architettura modulare della pipeline di Machine Learning

Il sistema è stato ingegnerizzato seguendo il principio della Singola Responsabilità (*Single Responsibility Principle* - SRP). L'architettura è composta da cinque macro-componenti indipendenti che comunicano in modo unidirezionale. Questo design permette di sostituire o aggiornare un singolo modulo (ad esempio, l'algoritmo di classificazione) senza rischiare di introdurre *side-effect* nelle procedure di estrazione o valutazione dei dati.

Il flusso di esecuzione e le responsabilità sono così orchestrati:

- **`config.py` (State & Configuration):** funge da gestore centralizzato della configurazione per l'intero applicativo. Definisce le costanti di sistema, i percorsi fisici delle directory (dati grezzi e risultati), le soglie di *preprocessing* e inizializza i dizionari contenenti i modelli di *Scikit-Learn* [8] e le configurazioni delle Reti Neurali.

- **`data_manager.py` (ETL Layer):** incapsula la logica di accesso al file system, la pulizia del segnale e la costruzione dei tensori. Gestisce internamente la complessità della cache binaria e garantisce la corretta separazione (*splitting*) tra esperimenti di *training* e di *test*.

- **`experiment_runner.py` (Orchestration Layer):** rappresenta il nucleo algoritmico del *framework*. Implementa il *pattern* della *Dependency Injection*,

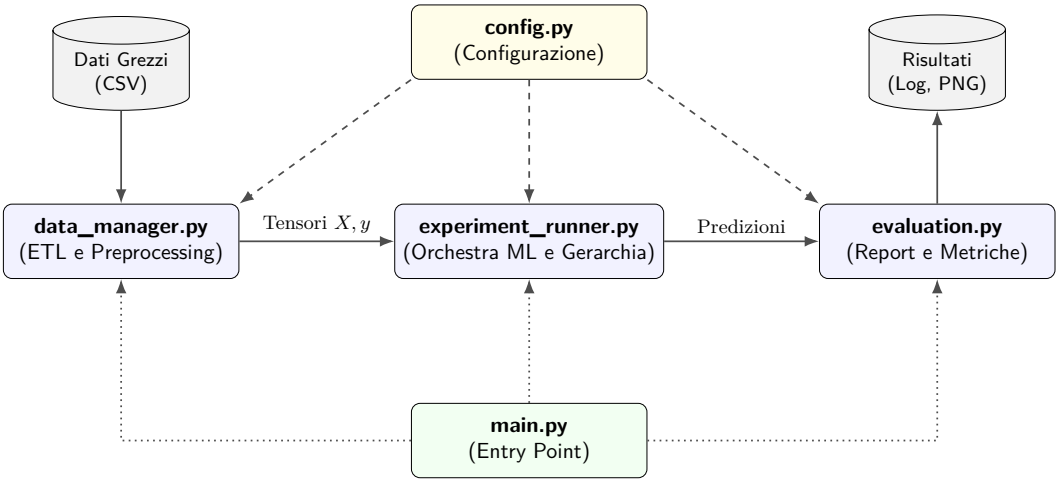


Figura 3.1. Diagramma a blocchi dell'architettura software e del flusso dei dati. Le frecce tratteggiate indicano le dipendenze di configurazione, quelle puntinate l'orchestrazione.

coordinando la *K-Fold Cross Validation*, applicando la logica di *Majority Voting* sugli esperimenti ed eseguendo l'addestramento dei modelli, sia *flat* che gerarchici.

- `evaluation.py` **(Presentation Layer):** gestisce esclusivamente la persistenza dei risultati e la reportistica visiva. Traduce le predizioni grezze in file di log formattati, graduatorie di merito (*Top 5*) e Matrici di Confusione ad alta risoluzione.

- `main.py` **(Entry Point):** rappresenta il punto di ingresso dell'applicazione. Si limita a istanziare i componenti principali e a richiamare le *suite* di test configurate, senza contenere logica algoritmica diretta.

Questa separazione garantisce che il ciclo di vita del dato sia lineare: acquisizione, trasformazione, inferenza e valutazione avvengono in moduli isolati, facilitando le operazioni di *debugging* e la validazione del software.

3.2 Il gestore dei dati: `DataManager`

Il modulo `DataManager` costituisce il livello di astrazione tra il *file system* e i motori di Machine Learning. La sua funzione principale consiste nell'acquisire i dati grezzi sensoriali, applicare le routine di *preprocessing* e fornire in *output* i tensori matematici (X, y) pronti per l'addestramento, astraendo la complessità logistica rispetto ai livelli superiori dell'architettura.

Per ottimizzare le prestazioni, in particolare considerando l'elevata frequenza di campionamento dei sensori, il modulo implementa un sistema di *caching* binario dinamico. Alla sua prima esecuzione, il sistema processa i file CSV grezzi e genera un archivio vettoriale serializzato (formato `.npy`). Nelle esecuzioni successive, il caricamento avviene direttamente dalla memoria *cache*, bypassando le operazioni di *parsing* testuale e riducendo significativamente i tempi di I/O (*Input/Output*).

Dal punto di vista della trasformazione del dato (*Data Engineering*), il modulo esegue un filtraggio iniziale del rumore ambientale tramite la sottrazione della *baseline* (aria ambientale di riferimento) e computa *feature* statistiche essenziali, quali media e varianza, sui singoli esperimenti.

Inoltre, il `DataManager` è responsabile della corretta segregazione dei dati durante le fasi di validazione incrociata. Esso gestisce dinamicamente la memoria degli esperimenti, assicurandosi che durante la generazione dei *fold* di *training* e *test*, le misurazioni appartenenti al medesimo esperimento fisico non vengano divise tra i due insiemi. Questa logica è fondamentale per prevenire fenomeni di *data leakage* (contaminazione dei dati), i quali comporterebbero una sovrastima ottimistica delle reali capacità diagnostiche dei classificatori.

3.3 Il motore di esecuzione: `ExperimentRunner`

Il componente `ExperimentRunner` rappresenta il nucleo algoritmico del *framework*. Agendo come orchestratore, il suo scopo è acquisire i tensori preparati dal `DataManager`, istanziare i modelli matematici definiti nella configurazione e gestire l'intero ciclo di validazione incrociata (*Cross-Validation*).

Per garantire una valutazione statistica rigorosa e non affetta da *overfitting*, il motore implementa una logica di *Group K-Fold Cross Validation* con un controllo stretto sulla provenienza dei dati: il partizionamento garantisce che i campioni appartenenti a un medesimo esperimento fisico non vengano mai utilizzati in parte per l'addestramento e in parte per il *test* all'interno dello stesso *fold*.

Le responsabilità principali del modulo sono così suddivise:

3.3.1 L'aggregazione delle predizioni: Majority Voting

Il motore esegue l'addestramento *frame-by-frame*, sfruttando l'elevata densità dei dati sensoriali. Tuttavia, per ottenere un'accuratezza sperimentale allineata alla realtà fisica, l'`ExperimentRunner` raccoglie le etichette predette ($\hat{y}$) prima della fase di valutazione statistica e applica una logica di *Majority Voting*.

Raggruppando i risultati tramite un identificativo univoco dell'esperimento (`ID_Esperimento`), il sistema estrae la moda statistica delle predizioni: la sostanza chimica che ottiene il maggior numero di voti (*frame* classificati) viene assegnata in modo univoco all'intero esperimento. Questa architettura emula il comportamento di un sistema di allarme reale, capace di ignorare fluttuazioni momentanee o rumore isolato in favore di una tendenza globale robusta.

3.3.2 Orchestrazione della Pipeline Gerarchica

Oltre alla validazione dei classificatori *flat* standard (Fase 1), l'`ExperimentRunner` implementa la logica di orchestrazione per la **Classificazione Gerarchica** (Fase 2). Questa procedura si articola in due stadi computazionali sequenziali:

1. **Stage 1 (Classificatore Binario):** i dati in ingresso vengono preliminarmente valutati da un modello addestrato a distinguere esclusivamente due macro-classi: "Idrocarburo" e "Non-Idrocarburo".

2. **Stage 2 (Specialista con Soglia):** solo i campioni classificati positivamente come idrocarburi vengono inoltrati al secondo stadio, dove sono sottoposti a un secondo modello specializzato (es. una Rete Neurale MLP ottimizzata). Se le probabilità generate dallo Stage 2 (*Softmax*) per una specifica classe (es. *Metano*) non superano una soglia di sicurezza minima configurabile, il modulo interviene riassegnando il risultato e rietichettandolo come "Non-Idrocarburo" (rigetto dei falsi positivi).

Le metriche di accuratezza (globale, isolata per lo Stage 1 e isolata per lo Stage 2) vengono poi compattate e inoltrate al modulo preposto alla reportistica.

Capitolo 4

Dettagli Tecnologici e Implementazione

4.1 Tecnologie utilizzate: Python e l'ecosistema Scikit-learn

In questo capitolo vengono descritti gli strumenti software, le librerie e le metodologie implementative adottate per la realizzazione del *framework*. La scelta si è orientata verso tecnologie *open-source* consolidate nella comunità scientifica, capaci di garantire portabilità, efficienza computazionale e un'ampia disponibilità di algoritmi allo stato dell'arte.

La scelta del linguaggio di programmazione è ricaduta su **Python**, in virtù della sua sintassi espressiva e della sua posizione di riferimento nel campo del *Data Science* e del *Machine Learning*. Python permette di gestire con versatilità sia operazioni di alto livello (come l'orchestrazione della logica gerarchica) sia operazioni a basso livello per la manipolazione di vettori numerici.

4.1.1 Il ruolo di Scikit-learn

La componente fondamentale della *pipeline* di classificazione è costituita da **Scikit-learn** [8], una delle librerie più diffuse per l'apprendimento automatico in Python. L'adozione di Scikit-learn ha garantito diversi vantaggi metodologici:

- **Standardizzazione dell'interfaccia:** grazie all'API coerente basata sui metodi `fit()` e `predict()`, è stato possibile implementare un sistema agnostico rispetto al modello utilizzato. Questo ha permesso di confrontare agevolmente algoritmi differenti (come *Random Forest* e *Multi-Layer Perceptrons*) all'interno del medesimo motore di esecuzione.

- **Strumenti di Model Selection:** la libreria fornisce i moduli necessari per la gestione della *K-Fold Cross Validation* e per il calcolo delle metriche di prestazione, garantendo rigore statistico nelle fasi di validazione.

- **Pre-elaborazione:** sono state impiegate le classi di *Scaling* e *Normalization* per proiettare i segnali dei sensori in intervalli numerici ottimali per l'adde-

stramento, evitando che *feature* con ordini di grandezza differenti potessero polarizzare i modelli.

4.1.2 L'ecosistema di supporto: NumPy e Pandas

L'efficienza del `DataManager` è strettamente legata all'impiego di **NumPy** e **Pandas**. *Pandas* è stata utilizzata per la fase di *Ingestion*, permettendo una manipolazione rapida dei file CSV forniti da Sensichips e la gestione dei metadati associati a ogni esperimento. *NumPy* rappresenta invece il fondamento della gestione tensoriale: la trasformazione dei dati in *array* multidimensionali ad alte prestazioni ha permesso di ottimizzare le operazioni algebriche necessarie per il *preprocessing* e di implementare il sistema di *caching* binario descritto in precedenza.

Infine, l'ambiente di sviluppo è stato isolato tramite *Virtual Environment*, assicurando la coerenza delle versioni tra le diverse macchine di test (Windows e Linux) e facilitando la replicabilità degli esperimenti mediante il file `requirements.txt`.

4.2 Algoritmi di classificazione: dai modelli classici a LightGBM e MLP

La complessità e la non-linearità dei segnali generati dai sensori hanno richiesto un'analisi comparativa tra diverse famiglie di algoritmi di classificazione. Durante le fasi preliminari del progetto, operando su dati grezzi non ancora raffinati (privi di estrazione avanzata di *feature*, bilanciamento o logiche di aggregazione per esperimento), è stata condotta un'indagine per identificare le architetture più promettenti, valutandole in termini di accuratezza e tempi di addestramento.

4.2.1 I limiti dei modelli classici e delle SVM

I modelli lineari e probabilistici tradizionali, quali *Logistic Regression*, *Linear Discriminant Analysis* e *Naive Bayes*, hanno mostrato prestazioni ridotte (con accuratezze spesso inferiori al 20% nei test preliminari), risultando inadeguati alla modellazione della sovrapposizione chimica tra le classi.

Un'attenzione particolare è stata dedicata alle *Support Vector Machines* (SVM), modelli geometrici fondati sulla teoria dell'apprendimento statistico [9] che ricercano l'iperpiano ottimo per massimizzare il margine di separazione tra le diverse classi. Sebbene teoricamente efficaci in spazi ad alta dimensionalità e supportate da implementazioni software storiche ed estremamente diffuse [10], all'atto pratico le SVM si sono rivelate del tutto inadeguate per questo specifico dominio applicativo. I tempi di convergenza sono risultati proibitivi (raggiungendo in alcuni scenari bilanciati oltre le 5 ore di calcolo per un singolo *fold*) e, in combinazione con la normalizzazione dei dati, i *kernel* lineari e gaussiani hanno frequentemente fallito la convergenza. Tali criticità prestazionali ne hanno decretato l'esclusione dalle fasi avanzate del framework.

4.2.2 L'efficienza del Gradient Boosting e degli alberi decisionali

Al contrario, le metodologie basate su insiemi di alberi decisionali (*Ensemble Learning*) hanno mostrato risultati rilevanti. Gli algoritmi di *Gradient Boosting* si basano sulla costruzione sequenziale di alberi, dove ogni nuovo albero mira a correggere gli errori predittivi commessi da quelli precedenti. Nello specifico, **XGBoost** [11] implementa questo paradigma ottimizzando rigorosamente l'esecuzione parallela e la regolarizzazione, mentre **LightGBM** [12] utilizza una strategia di crescita asimmetrica delle foglie (*leaf-wise*) per ridurre drasticamente i tempi di calcolo e l'utilizzo di memoria.

Nei test preliminari, questi algoritmi hanno registrato tempi di addestramento nell'ordine di poche decine di secondi, con un'accuratezza prossima al 48%, performando quasi al livello delle reti neurali sui dati non processati. Questa efficienza li ha resi delle *baseline* robuste e intrinsecamente resistenti al rumore parametrico.

4.2.3 La predominanza del Multi-Layer Perceptron (MLP)

L'ultima famiglia analizzata, divenuta poi il fulcro dell'architettura finale, è stata quella delle Reti Neurali Artificiali [13, 14], nello specifico i **Multi-Layer Perceptron (MLP)** [15]. Sono state testate e messe a confronto diverse topologie di rete, variando la profondità e il numero di neuroni nei layer nascosti (come architetture "a diamante" `64-128-64`, reti profonde `256-128-64` o architetture piatte).

Se nelle primissime esecuzioni su dati grezzi le performance degli MLP viaggiavano in parallelo con quelle dei modelli di *Gradient Boosting*, l'evoluzione della pipeline ha cambiato drasticamente gli equilibri. Con l'introduzione progressiva di logiche di aggregazione per esperimento (*Majority Voting*), l'applicazione di pesi per il bilanciamento delle classi e un *preprocessing* mirato, combinati all'impiego di metodi avanzati di ottimizzazione stocastica per l'aggiornamento dei gradienti [16], ha iniziato a emergere in modo netto la superiorità delle reti neurali.

Da quel punto in poi, le architetture MLP hanno dominato le metriche di valutazione, riuscendo a mappare le relazioni non lineari latenti dei sensori MOX e capacitivi in modo molto più profondo rispetto agli alberi decisionali. Questa predominanza li ha resi i candidati ideali non solo per la classificazione standard, ma soprattutto per agire come specialisti ad alta precisione all'interno dello *Stage 2* del modello gerarchico.

4.3 Implementazione del preprocessing e normalizzazione dei segnali

Le prestazioni di un modello di Machine Learning, con particolare riferimento alle Reti Neurali Artificiali, dipendono strettamente dalla qualità dei dati in ingresso. I segnali grezzi acquisiti dai sensori MOX e capacitivi presentano ordini di grandezza eterogenei e sono influenzati dal rumore di fondo dovuto alle misurazioni di impedenza a diverse frequenze e temperature. Per mitigare tali effetti, all'interno del modulo `DataManager` è stata implementata un'articolata *pipeline* di pre-elaborazione matematica.

4.3.1 Sottrazione della Baseline Ambientale

Una criticità primaria nella rilevazione dei gas è rappresentata dalla variabilità dell'ambiente circostante. Per isolare il segnale specifico della sostanza target, il sistema applica una procedura di *Baseline Subtraction*. Utilizzando le misurazioni di riferimento dell'aria pura acquisite preliminarmente a ogni esperimento, il `DataManager` calcola la differenza tra il campione attuale e la *baseline* ambientale. Questa operazione permette di filtrare le derive lente del sensore (*drift*) e le fluttuazioni termiche o igrometriche non correlate alla sostanza in esame, restituendo un segnale che rappresenta la variazione indotta esclusivamente dal gas.

4.3.2 Estrazione delle Feature

Invece di impiegare solamente le singole letture istantanee, potenzialmente influenzate da *white noise* e picchi anomali, il sistema estrae *feature* statistiche dai *frame* di misurazione. Questa estrazione è stata implementata in modo *custom* sfruttando le operazioni vettoriali della libreria NumPy[1]. In linea con le metodologie standard di estrazione delle feature in ambito chemo-sensoriale [17], per ogni istante di campionamento (ovvero per ogni riga della matrice dei dati), il sistema calcola la media, la deviazione standard e il valore massimo registrati trasversalmente su tutti i sensori attivi. Queste tre nuove variabili matematiche vengono concatenate alle misurazioni grezze, fornendo una rappresentazione del dato più stabile e densa di informazioni, migliorando così il rapporto segnale/rumore (SNR) prima ancora della fase di addestramento.

4.3.3 Scaling e Normalizzazione (StandardScaler)

Il passaggio conclusivo nella preparazione dei tensori (X) è la normalizzazione. Poiché l'array di sensori restituisce valori di impedenza e fase che variano su scale numeriche molto distanti, l'assenza di un corretto *scaling* introdurrebbe un *bias* computazionale: le *feature* con magnitudo superiore dominerebbero la funzione di costo durante la *Backpropagation* del Multi-Layer Perceptron.

Per ovviare a questo problema, è stato integrato lo `StandardScaler` di *Scikit-learn*, che standardizza le *feature* rimuovendo la media e ridimensionando la varianza a un valore unitario (distribuzione normale con $\mu = 0$ e $\sigma = 1$). Come confermato dalla validazione empirica, l'applicazione congiunta della sottrazione della *baseline* e della normalizzazione statistica è risultata determinante per garantire la convergenza dei modelli basati su gradiente e per ottimizzare le capacità predittive delle architetture MLP.

4.4 Gestione della configurazione centralizzata: `config.py`

Uno dei principi metodologici dell'Ingegneria del Software applicata al Machine Learning è la separazione tra la logica esecutiva (il nucleo algoritmico) e la parametrizzazione (le variabili di dominio). Nel *framework* sviluppato, questo paradigma, noto

[1] L'implementazione algoritmica dettagliata della generazione vettoriale delle feature è consultabile all'interno del modulo `data_manager.py` disponibile nella repository GitHub del progetto.

come *Configuration over Code*, è stato implementato tramite il modulo centralizzato `config.py`.

Questo file funge da gestore centralizzato della configurazione per l'intero applicativo, consentendo a ricercatori e sviluppatori di modificare il comportamento del sistema senza la necessità di intervenire sui moduli esecutivi (come `DataManager` o `ExperimentRunner`), prevenendo alterazioni accidentali del codice logico.

Il modulo centralizza tre macro-aree di configurazione:

4.4.1 Definizione dell'Ambiente e dei Target Chimici

Il file gestisce in modo agnostico i percorsi di sistema (mediante le librerie `os` e `pathlib`), garantendo che le *directory* per i dati di *training* e per la *cache* vettoriale (`EXTRACTED_DATA.npy`) vengano risolte correttamente a prescindere dal sistema operativo *host*. Inoltre, ospita le strutture dati fondamentali per la classificazione: la lista completa delle 25 sostanze target (`COMPONENTS`) e, in particolare, la sottolista dedicata agli idrocarburi (`HYDROCARBONS`: metano, butano, benzina, fluido per accendini, diesel, cherosene). Quest'ultima funge da discriminante matematica per l'instradamento (*routing*) dei tensori all'interno dell'architettura gerarchica (Stage 1 e Stage 2).

4.4.2 Iniezione delle Dipendenze (Dependency Injection) e Duck Typing

Una caratteristica rilevante del modulo è la pre-istanziazione dei modelli di *Scikit-learn* e *LightGBM*. In Ingegneria del Software, la *Dependency Injection* è un *design pattern* in cui un oggetto riceve le sue dipendenze dall'esterno anziché crearle al proprio interno. Invece di delegare la creazione degli algoritmi ai moduli esecutivi, `config.py` istanzia direttamente le classi (es. `RandomForestClassifier`, `MLPClassifier`) con i relativi iperparametri predefiniti (come il *learning rate*, il numero di stimatori o le funzioni di attivazione).

Queste istanze vengono poi raggruppate in dizionari semantici (es. `FAST_CLASSIFIERS`, `STAGE1_CLASSIFIERS`) che il motore principale importa e utilizza dinamicamente. Il modulo orchestratore accetta questi modelli fidandosi unicamente della presenza dei metodi standard `fit()` e `predict()`, applicando un principio noto in Python come *duck typing*: il codice non verifica rigidamente la classe esatta dell'oggetto passato, ma si limita a constatare che esso possieda i comportamenti richiesti per l'addestramento e l'inferenza.

4.4.3 Progettazione Topologica per Reti Neurali

Un'ampia sezione del file è dedicata alla definizione delle architetture (*Architecture Search*) per i Multi-Layer Perceptron. Sono definite liste di *tuple* che rappresentano decine di topologie di rete differenti, categorizzate per struttura logica:

- **A imbuto (Funnel):** strutture decrescenti (es. `256, 128, 64`) per forzare una compressione delle *feature*.

- **A diamante (Diamond):** strutture con espansione centrale (es. `64, 128, 64`) per proiettare i dati in uno spazio dimensionale superiore prima della classificazione.
- **A larghezza costante (Cylinder/Flat):** strutture con livelli (*layer*) di dimensioni invariate.

Questa elevata modularità ha permesso di automatizzare l'esplorazione sistematica delle reti neurali, rendendo l'individuazione della topologia ottimale (come l'architettura a diamante risultata vincente) un processo *data-driven* e facilmente replicabile.

Capitolo 5

Sperimentazione e Validazione

Questo capitolo presenta i risultati ottenuti dalla *pipeline* di classificazione, descrivendo la metodologia di test e analizzando criticamente le prestazioni dei modelli. L'analisi si articola partendo dalla validazione dei modelli *flat* (Fase 1) fino ad arrivare ai risultati specialistici dell'architettura gerarchica (Fase 2).

5.1 Protocollo sperimentale e K-Fold Cross Validation

Per valutare in modo sistematico le prestazioni dei modelli ed escludere che i risultati siano frutto di fluttuazioni statistiche momentanee o di fenomeni di *overfitting*, il *framework* è stato sottoposto a un definito protocollo sperimentale. Tutti i test finali discussi di seguito sono stati eseguiti sulla versione ottimizzata dei tensori: dati sottoposti a estrazione delle *feature*, sottrazione della *baseline* ambientale e normalizzazione statistica.

5.1.1 La validazione incrociata e l'isolamento degli esperimenti

La stima delle reali capacità di generalizzazione del sistema è stata affidata a una **10-Fold Cross Validation**. A differenza della classica validazione incrociata che mescola e divide i singoli campioni (*frame*) in percentuali fisse (es. 90/10), il *framework* opera una segregazione a livello macroscopico basata sull'identificativo fisico della misurazione (*Group K-Fold*).

In ogni iterazione, la totalità dei dati appartenenti a un intero esperimento fisico (corrispondente a uno specifico file CSV originario) viene isolata e impiegata esclusivamente come *test set*, mentre i dati di tutti gli altri esperimenti costituiscono il *training set*. Questo approccio svolge una duplice funzione: da un lato previene efficacemente la contaminazione dei dati (*data leakage*), dall'altro fornisce la base logica ideale per l'applicazione del *Majority Voting*, permettendo di valutare la robustezza predittiva del modello su una sequenza di misurazioni fisiche totalmente inedita.

5.1.2 Strategia di Bilanciamento (Hybrid Resampling)

Un problema intrinseco nei *dataset* sensoriali è il forte sbilanciamento delle classi (ad esempio, le misurazioni prolungate dell'aria o del vapore acqueo sono fisiologicamente

molto più numerose rispetto a quelle dei gas target concentrati), una criticità nota in letteratura per il suo impatto negativo sulle prestazioni dei modelli di classificazione [18]. Fornire a un classificatore un *dataset* sbilanciato comporterebbe lo sviluppo di un *bias* verso la classe maggioritaria.

Per ovviare a questo problema, rispetto alle tradizionali tecniche di oversampling sintetico come SMOTE [19], durante la costruzione del *training set* in ogni *fold* è stata implementata una logica algoritmica *custom* di bilanciamento ibrido (*Hybrid Resampling*):

1. **Calcolo del target:** il sistema calcola la cardinalità media (il numero di campioni) tra tutte le 25 classi chimiche presenti. Questo valore numerico funge da *target* di bilanciamento ottimale.
2. **Undersampling:** per le classi maggioritarie (il cui numero di istanze supera la media matematica), viene applicato un sottocampionamento casuale senza reimmissione (*without replacement*), scartando i dati in eccesso.
3. **Oversampling:** per le classi minoritarie, viene applicato un sovracampionamento con reimmissione (*with replacement*), duplicando casualmente i campioni esistenti fino a raggiungere esattamente il *target*.

Questa strategia algoritmica induce i modelli (in particolare gli *MLP*) ad assegnare lo stesso peso a ogni sostanza chimica nel calcolo della funzione di costo, requisito essenziale per la rilevazione affidabile di gas critici.

5.1.3 Metriche di Valutazione

Al termine delle 10 iterazioni, il modulo valutatore aggrega i risultati. La metrica quantitativa primaria scelta per la valutazione matematica dei modelli è l'**Accuratezza** (*Accuracy*), che definisce la percentuale di predizioni corrette sul totale delle valutazioni effettuate. Nello specifico, essendo calcolata a valle del *Majority Voting*, l'Accuracy quantifica il rapporto tra il numero di esperimenti fisici classificati correttamente (in cui la moda dei *frame* coincide con la sostanza chimica realmente rilasciata) e il numero totale di esperimenti presenti nel *test set*. Tale metrica viene riportata in forma aggregata: valore medio affiancato dalla relativa Deviazione Standard.

L'analisi qualitativa e diagnostica (la valutazione della distribuzione degli errori, dei falsi positivi e dei falsi negativi) è stata invece demandata all'ispezione visiva delle Matrici di Confusione.

5.2 Analisi dei risultati per modelli standard (Fase 1)

Il primo obiettivo della fase di sperimentazione è stato valutare la capacità del sistema di classificare correttamente l'intero spettro delle 25 classi chimiche (Fase 1 - *Flat Classification*). Come anticipato nell'analisi tecnologica (Capitolo 4), i test preliminari condotti su dati grezzi, privi di aggregazione per esperimento, avevano mostrato limitazioni significative: i modelli basati su alberi (*LightGBM*, *Random Forest*) faticavano a superare il 50% di accuratezza e le reti neurali si assestavano su valori affini.

5.2.1 L'impatto della Pipeline di Preprocessing e del Majority Voting

L'attivazione della *pipeline* definitiva ha inciso in modo determinante sulle prestazioni predittive. L'azione combinata della normalizzazione statistica (`StandardScaler`) e della strategia *custom* di *Hybrid Resampling* ha permesso ai modelli basati sul calcolo del gradiente di convergere verso minimi globali maggiormente stabili. Tuttavia, il fattore principale di incremento prestazionale è derivato dall'introduzione del *Majority Voting*. Valutare le predizioni estraendo la moda statistica sull'intera finestra dell'esperimento fisico, anziché sul singolo *frame* istantaneo, ha permesso di filtrare efficacemente il rumore isolato e i picchi anomali del sensore, restituendo un verdetto unificato e robusto.

5.2.2 La topologia ottimale: MLP (512, 256)

Sui dati così elaborati, le reti neurali artificiali hanno dimostrato prestazioni marcatamente superiori rispetto agli algoritmi di *Ensemble Learning*, confermandosi come gli unici modelli capaci di mappare in modo compiuto l'elevata non-linearità della *cross*-sensitività dei sensori.

Al termine della *10-Fold Cross Validation*, il modello più performante è risultato essere un **Multi-Layer Perceptron** caratterizzato da un'architettura profonda "a imbuto", composta da due *layer* nascosti rispettivamente di **512 e 256 neuroni**. I migliori iperparametri individuati per questa topologia sono stati la funzione di attivazione `ReLU` e un fattore di regolarizzazione L2 (`alpha`) pari a `0.01`, abbinati all'ottimizzatore *Adam*.

Questo modello ha raggiunto un'**Accuratezza media del 94.6%**, con una **Deviazione Standard del ± 4.9%** calcolata sui 10 *fold*.

Ottenere un'accuratezza prossima al 95% su un problema a elevata cardinalità (25 classi, comprendenti molecole chimicamente affini e disturbi ambientali) costituisce un risultato di significativo valore ingegneristico. La deviazione standard contenuta (± 4.9%) certifica, inoltre, che la rete non ha subìto fenomeni di memorizzazione specifica dei campioni (*overfitting*), ma ha generalizzato con successo i *pattern* distintivi dei gas anche sui file CSV di *test* mai osservati durante la fase di addestramento.

5.3 Valutazione delle prestazioni gerarchiche (Fase 2)

Nonostante le elevate prestazioni ottenute nella classificazione *flat* (94.6%), la gestione delle criticità legate alla sicurezza ha imposto un'evoluzione architetturale. Come delineato in fase di analisi dei requisiti, in uno scenario industriale è inaccettabile confondere un idrocarburo infiammabile con una sostanza innocua (Falso Negativo), mentre è tollerabile un margine di errore superiore tra gas a basso rischio. Per minimizzare le misclassificazioni critiche, è stata implementata e validata l'architettura gerarchica a due stadi.

5.3.1 Stage 1: Il Filtro Binario (LightGBM)

Il primo stadio dell'architettura agisce da filtro primario, separando gli idrocarburi dalle restanti sostanze. Per questo compito, il modello ottimale è risultato essere una configurazione ad alta sensibilità di **LightGBM** [12]. Sfruttando un forte sbilanciamento artificiale dei pesi interni (`scale_pos_weight = 15.0`), questo algoritmo ha massimizzato la *Recall* sulla classe positiva, raggiungendo un'accuratezza mirata sugli idrocarburi del **96.1%** (con una deviazione del $\pm$ 9.3%). Questo risultato garantisce un tasso di Falsi Negativi pari a solo il 3.9%, assicurando che la quasi totalità dei gas pericolosi superi il filtro e venga inoltrata allo stadio successivo.

5.3.2 Stage 2: Lo Specialista (MLP 64-256-64)

I campioni classificati positivamente dallo Stage 1 vengono passati al secondo livello, dove un modello specializzato ha il compito di identificare la molecola specifica (es. metano, benzina, cherosene). Tra le topologie valutate, il modello che ha mostrato le prestazioni ottimali è un **MLP con architettura "a diamante" (64, 256, 64)**, `alpha=0.01` e attivazione `ReLU`. La scelta di questa struttura non è stata determinata unicamente dal picco di accuratezza assoluta, ma dalla sua notevole **stabilità**: a differenza di altre reti (come quelle a imbuto o a larghezza costante) che subivano marcate fluttuazioni prestazionali al variare degli iperparametri, l'architettura a diamante ha mantenuto risultati solidi in tutte le configurazioni di validazione, dimostrando una capacità di generalizzazione superiore.

5.3.3 Meccanismo di Rigetto e Ottimizzazione della Soglia

L'elemento conclusivo dell'architettura gerarchica è il meccanismo di scarto basato sulle probabilità (*Softmax*) emesse dallo Stage 2. È stata condotta un'analisi di sensibilità per individuare la soglia di confidenza ideale, testando valori nell'intervallo [0.4, 0.8]. L'ottimo paretiano è stato individuato al valore di **0.7 (70%)**.

- **Soglie inferiori (< 0.7):** rendevano il sistema eccessivamente permissivo, consentendo a sostanze innocue (Falsi Positivi sfuggiti allo Stage 1) di essere confermate come idrocarburi, generando così falsi allarmi industriali.
- **Soglie superiori (> 0.7):** rendevano il sistema troppo conservativo, inducendo il rigetto di reali campioni di gas pericolosi (riassegnati forzatamente alla classe "Non-Idrocarburo"). Questa dinamica reintroduce falsi negativi critici, vanificando il lavoro di massimizzazione della *Recall* effettuato dallo Stage 1.

In sintesi, la configurazione finale atta a soddisfare i requisiti di sicurezza del progetto si articola nella seguente sequenza logica: **LightGBM (Stage 1)** $\rightarrow$ **MLP 64-256-64 (Stage 2)** $\rightarrow$ **Soglia di rigetto a 0.7**.

5.4 Analisi visiva tramite Matrici di Confusione

Per un'analisi dettagliata del comportamento dei modelli, oltre alle metriche numeriche aggregate, è stata condotta una valutazione visiva mediante le Matrici di

Confusione. Questo strumento permette di individuare la natura degli errori commessi, evidenziando quali classi di gas presentano le maggiori somiglianze spettrali per l'array di sensori.

5.4.1 Matrice di Confusione - Fase 1 (Classificazione Flat)

La Figura 5.1 mostra le prestazioni del modello MLP (512, 256) sull'intero spettro delle 25 classi. Si osserva un'evidente concentrazione sulla diagonale principale, a conferma dell'accuratezza globale ottenuta (94.6%) e dell'efficacia del *Majority Voting*; numerose sostanze, tra cui ammoniaca e idrossido di sodio, risultano classificate con una precisione del 100%.

Tuttavia, l'analisi dei valori fuori diagonale rivela dinamiche di *cross*-sensitività complesse. Ad esempio, si nota una marcata confusione unidirezionale tra l'aria ambientale (*AIR*) e il cherosene (*KEROSENE*), con circa il 40% delle istanze di aria erroneamente classificate come idrocarburo. Si registrano inoltre sovrapposizioni tra solventi e carburanti, come nel caso della *Gasoline* confusa con l'alcol *Ethanol*.

Queste specifiche tipologie di errore, in particolare la generazione di falsi positivi su sostanze potenzialmente pericolose a partire da misurazioni innocue, giustificano l'adozione dell'architettura gerarchica con meccanismo di rigetto (Fase 2), necessaria per sopprimere i falsi allarmi e garantire l'affidabilità del sistema in contesti operativi.

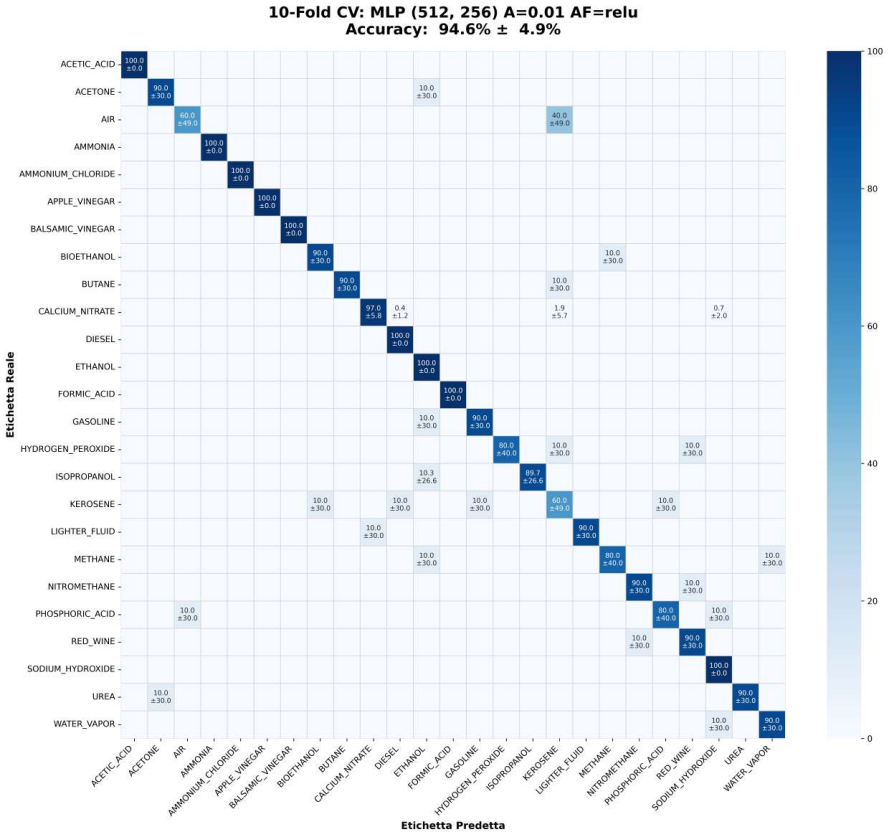


Figura 5.1. Matrice di confusione per la classificazione a 25 classi (Fase 1).

5.4.2 Matrice di Confusione - Fase 2 (Stage 2 Gerarchico)

La Figura 5.2 illustra i risultati dello Stage 2 dell'architettura gerarchica, focalizzato sulla discriminazione specifica degli idrocarburi. In questo caso, la matrice evidenzia l'elevata selettività del modello specializzato (MLP 64-256-64) e l'efficacia della soglia di rigetto impostata a 0.7. La quasi totale assenza di falsi negativi (idrocarburi classificati come non pericolosi) conferma l'efficacia della strategia gerarchica proposta per scopi di sicurezza industriale.

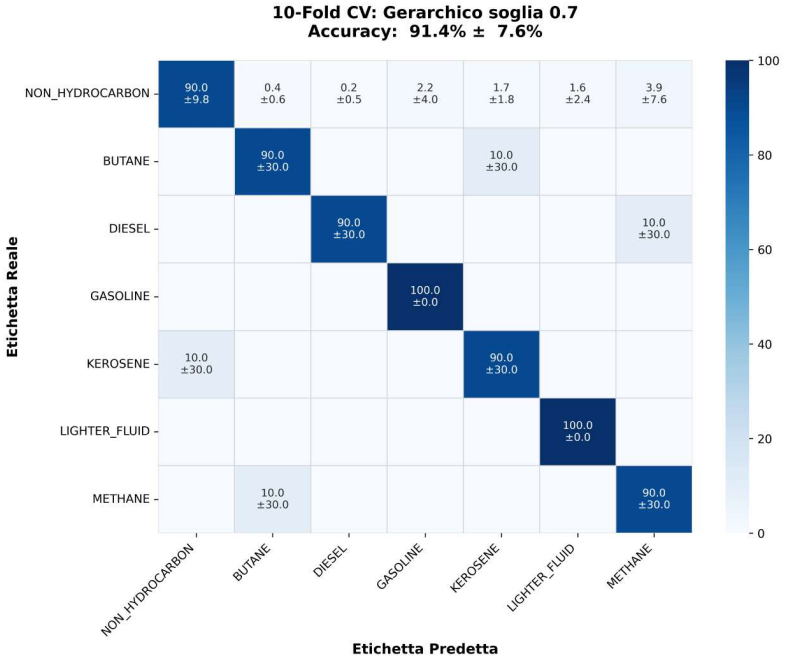


Figura 5.2. Matrice di confusione dello Stage 2 per l'identificazione specifica degli idrocarburi.

Capitolo 6

Conclusioni e Sviluppi Futuri

6.1 Sintesi del lavoro e raggiungimento degli obiettivi

Il presente lavoro di tesi si era prefissato l'obiettivo di progettare, ingegnerizzare e validare una *pipeline* di Machine Learning in grado di decodificare i complessi segnali provenienti da una matrice di sensori gas (MOX e capacitivi), superando le criticità intrinseche legate alla *cross*-sensitività chimica e al rumore ambientale. Alla luce dei risultati sperimentali ottenuti, gli scopi del progetto risultano raggiunti, portando allo sviluppo di un *framework* software robusto, modulare e orientato ad applicazioni industriali.

Il percorso di ricerca ha confermato che l'applicazione diretta di algoritmi di classificazione sui dati sensoriali grezzi restituisce risultati insufficienti (con accuratezze inferiori al 50%). Il fattore determinante dell'architettura proposta risiede nell'articolata fase di *Data Engineering*: l'integrazione della sottrazione della *baseline*, della normalizzazione statistica e della logica di aggregazione per esperimento tramite *Majority Voting* su gruppi di *fold* isolati, ha permesso di trasformare misurazioni fortemente rumorose in predizioni stabili e affidabili.

Il raggiungimento degli obiettivi si è concretizzato attraverso le due macro-fasi previste:

- **Validazione della Classificazione Multi-classe (Fase 1):** l'ottimizzazione di un Multi-Layer Perceptron profondo (topologia 512-256), supportato da una strategia algoritmica *custom* di *Hybrid Resampling* per il bilanciamento delle classi, ha permesso di mappare lo spazio ad altissima dimensionalità dei sensori. Il raggiungimento di un'accuratezza globale del 94.6% sul riconoscimento simultaneo di 25 sostanze chimiche e condizioni ambientali rappresenta un risultato scientifico significativo, dimostrando le elevate capacità predittive dell'hardware fornito da Sensichips [1].

- **Ottimizzazione della Sicurezza tramite Architettura Gerarchica (Fase 2):** per rispondere a stringenti requisiti di sicurezza industriale, si è superato un approccio monolitico in favore di un design modulare. L'adozione di un classificatore binario *LightGBM* ad alto sbilanciamento come "filtro" di primo livello ha minimizzato il tasso di Falsi Negativi sulle molecole infiammabili (idrocarburi) al 3.9%. Parallelamente, l'introduzione di una soglia di confidenza

al 70% sul modello specialista di secondo livello (MLP 64-256-64) ha fornito al sistema un robusto meccanismo di rigetto, sopprimendo i falsi allarmi derivanti da misurazioni innocue come l'aria ambientale.

Oltre ai traguardi prettamente algoritmici, un ulteriore risultato del progetto è la realizzazione di un *software* ingegnerizzato secondo i migliori standard industriali (PEP 8, *Type Hinting*, *Single Responsibility Principle*). La centralizzazione dei parametri nel modulo `config.py` fornisce ai ricercatori uno strumento facilmente manutenibile, scalabile e immediatamente riutilizzabile per future campagne di acquisizione dati.

6.2 Sviluppi Futuri

Sebbene i risultati ottenuti confermino la validità dell'approccio proposto, il lavoro svolto costituisce una base metodologica per ulteriori evoluzioni tecnologiche. Di seguito vengono delineati i principali percorsi di ricerca futuri:

- **Analisi di Miscele Chimiche:** il sistema attuale è configurato per l'identificazione di composti singoli in condizioni controllate. Una possibile estensione del progetto consiste nell'espansione del *dataset* a miscele complesse (es. presenza simultanea di metano e monossido di carbonio). Tale evoluzione richiederà il passaggio da una classificazione *multi-class* a una logica *multi-label*, finalizzata alla stima delle concentrazioni relative dei singoli componenti in presenza di interferenze chimiche incrociate.
- **Implementazione su Dispositivi Edge e Real-time:** per l'integrazione in contesti operativi, è previsto il *porting* degli algoritmi su *hardware* dedicato a bassa potenza (es. microcontrollori ARM o sistemi *embedded*). Questo passaggio richiederà l'ottimizzazione del codice e l'adozione di tecniche di *Model Quantization* e *Pruning* per ridurre il carico computazionale delle reti neurali, consentendo l'esecuzione dell'inferenza in tempo reale direttamente a bordo del sistema di acquisizione.
- **Auto-calibrazione e Compensazione del Drift:** i sensori chimici sono soggetti a fenomeni di deriva nel lungo periodo (*drift*) legati all'invecchiamento dei materiali sensibili. Lo sviluppo di moduli di apprendimento continuo (*Continual Learning*) permetterebbe l'adattamento automatico del sistema a tali variazioni, limitando la necessità di interventi periodici di ricalibrazione manuale.

Bibliografia

[1] Sensichips s.r.l. *SCA Air Sensor - Technical Reference Manual*, 2025. Documentazione tecnica e specifiche dell'array di sensori MOX e capacitivi.

[2] Julian W Gardner and Philip N Bartlett. A brief history of electronic noses. *Sensors and Actuators B: Chemical*, 18(1-3):211–220, 1994.

[3] Andreas Schütze, T Baur, M Leidinger, W Reimringer, L Spinelle, and M Gerboles. Highly sensitive and selective voc sensor systems based on semiconductor gas sensors: How to? *Micromachines*, 8(3):79, 2017.

[4] Ghenadii Korotcenkov. Metal oxides for solid-state gas sensors: What determines our choice? *Materials Science and Engineering: B*, 139(1):1–23, 2007.

[5] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *nature*, 521(7553):436–444, 2015.

[6] Dimitri Bertsekas. *Reinforcement learning and optimal control*. Athena Scientific, 2019.

[7] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Foster Provost. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 6(4):1–21, 2012.

[8] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[9] Vladimir Vapnik. *The nature of statistical learning theory*. Springer science & business media, 2013.

[10] Chih-Chung Chang and Chih-Jen Lin. Libsvm: A library for support vector machines. *ACM transactions on intelligent systems and technology (TIST)*, 2(3):1–27, 2011.

[11] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[12] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*. 2017.

[13] Christopher M Bishop. *Neural networks for pattern recognition*. Oxford university press, 1995.

[14] Simon Haykin. *Neural networks: a comprehensive foundation*. Prentice Hall PTR, 1994.

[15] Luis B Almeida. Multilayer perceptrons. In *Handbook of neural computation*, pages C1–2. CRC Press, 2020.

[16] Léon Bottou, Frank E Curtis, and Jorge Nocedal. Optimization methods for large-scale machine learning. *SIAM review*, 60(2):223–311, 2018.

[17] Alexander Vergara, Shankar Vembu, Tuba Ayhan, Margaret A Ryan, Margie L Homer, and Ramón Huerta. Chemical gas sensor drift compensation using classifier ensembles. *Sensors and Actuators B: Chemical*, 166:320–329, 2012.

[18] Haibo He and Edwardo A Garcia. Learning from imbalanced data. *IEEE Transactions on knowledge and data engineering*, 21(9):1263–1284, 2009.

[19] Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. Smote: synthetic minority over-sampling technique. *Journal of artificial intelligence research*, 16:321–357, 2002.